

RESOLUCION DE EJERCICIOS SEMANA 11

Estudiante: Cuba Villanueva Javier Amilcar

TEMA 1: Proyecto 1 - Implementación de acceso mixto

1. **Enunciado del ejercicio** La Universidad requiere que los trabajadores internos accedan mediante autenticación Windows, mientras que los consultores externos utilicen autenticación SQL Server. Diseñe e implemente una estrategia de autenticación para la base de datos GradosTitulos que permita administrar ambos tipos de acceso.
2. Script de la solución en SQL

```
1  USE master;
2  GO
3
4  CREATE LOGIN [UNIVERSIDAD\PersonalInterno] FROM WINDOWS;
5  GO
6
7  CREATE LOGIN ConsultorExterno
8      WITH PASSWORD = 'PasswordSeguro2026#',
9      CHECK_POLICY = ON,
10     CHECK_EXPIRATION = ON;
11  GO
12
13  USE GradosTitulos;
14  GO
15
16  CREATE USER [UNIVERSIDAD\PersonalInterno] FOR LOGIN [UNIVERSIDAD\PersonalInterno];
17  CREATE USER ConsultorExterno FOR LOGIN ConsultorExterno;
18  GO
```

3. **Justificación técnica de la solución** El script se ejecuta primero en la base de datos del sistema master para crear los inicios de sesión (Logins) a nivel de instancia. Para el personal interno, se delega la autenticación a Active Directory (Windows). Para el consultor, se crea un login nativo de SQL Server forzando las directivas de seguridad locales. Luego, se cambia el contexto a GradosTitulos para mapear estos logins como usuarios (Users) dentro de la base de datos.
4. **Explicación de las buenas prácticas utilizadas**
 - **Separación de Instancia y Base de Datos:** Se diferencia claramente la creación del Login (master) y la del User (GradosTitulos).
 - **Directivas de Contraseña:** Se habilitan CHECK_POLICY y CHECK_EXPIRATION para los usuarios de SQL Server, garantizando que el consultor externo cumpla con la longitud, complejidad y rotación de contraseñas exigidas por el sistema operativo.

TEMA 1: Proyecto 2 - Gestión de accesos para miembros del Jurado

1. **Enunciado del ejercicio** Los miembros del jurado necesitan acceder únicamente a información de tesis y sustentaciones. Configure accesos diferenciados utilizando autenticación Windows y SQL.
2. Script de la solución en SQL

```

11 USE GradosTitulos;
12 GO
13
14 CREATE ROLE RolJurados;
15 GO
16
17 GRANT SELECT ON Tramite.TrabajoInvestigacion TO RolJurados;
18 GRANT SELECT ON Evaluacion.Sustentacion TO RolJurados;
19 GRANT SELECT ON Evaluacion.V_NotaFinalSustentacion TO RolJurados;
20 GO
21
22 ALTER ROLE RolJurados ADD MEMBER [UNIVERSIDAD\PersonalInterno];
23 ALTER ROLE RolJurados ADD MEMBER ConsultorExterno;
24 GO

```

3. Justificación técnica de la solución En lugar de otorgar permisos tabla por tabla a cada usuario de forma individual, se crea un rol de base de datos personalizado llamado RolJurados. A este rol se le conceden permisos de lectura (SELECT) estrictamente sobre las tablas y vistas requeridas (TrabajoInvestigacion y Sustentacion). Finalmente, se añaden los usuarios (tanto de Windows como de SQL) a este rol.

4. Explicación de las buenas prácticas utilizadas

- **Principio de Mínimo Privilegio:** Los jurados solo reciben permiso de SELECT, previniendo modificaciones accidentales o malintencionadas en los registros académicos.
- **Gestión Basada en Roles (RBAC):** Asignar permisos a un rol en lugar de a usuarios específicos facilita enormemente la escalabilidad y el mantenimiento administrativo.

TEMA 1: Proyecto 3 - Gestión de accesos para miembros del Jurado de Tesis

- 1. Enunciado del ejercicio** La Oficina de Tecnologías de Información requiere aplicar políticas estrictas para las cuentas SQL Server que administran los trámites de grados y títulos.
- 2. Script de la solución en SQL**

```

1  USE master;
2  GO
3
4
5  -- Creación de un login administrativo con políticas estrictas de seguridad
6  CREATE LOGIN AdminTramitesGT
7      WITH PASSWORD = 'TempPassword2026!#',
8      CHECK_POLICY = ON,
9      CHECK_EXPIRATION = ON,
10     MUST_CHANGE = ON;
11 GO
12
13 USE GradosTitulos;
14 GO
15
16 -- Mapeo del login a un usuario en la base de datos
17 CREATE USER AdminTramitesGT FOR LOGIN AdminTramitesGT;
18 GO

```

3. Justificación técnica de la solución Se utilizan los argumentos nativos de seguridad de T-SQL para inicios de sesión. CHECK_POLICY = ON obliga a que la contraseña cumpla con las directivas de complejidad de Windows (longitud, caracteres especiales). CHECK_EXPIRATION = ON fuerza la rotación periódica de la contraseña. MUST_CHANGE = ON es una medida crítica que obliga al administrador a cambiar esta contraseña temporal en su primer inicio de sesión.

4. Explicación de las buenas prácticas utilizadas

- **Delegación de Políticas:** En lugar de crear reglas personalizadas complejas en SQL Server, se delega la validación a las políticas locales del sistema operativo, lo que centraliza la seguridad.
- **Credenciales de un solo uso inicial (MUST_CHANGE):** Garantiza que ni siquiera el DBA que ejecuta el script conozca la contraseña final del administrador de trámites, asegurando el principio de no repudio.

TEMA 2: Proyecto 1 - Configuración segura de servicios SQL Server

1. **Enunciado del ejercicio** La base de datos GradosTitulos será utilizada en producción. Configure cuentas de servicio específicas para SQL Server y SQL Server Agent.
2. Script de la solución en SQL

```
-- 4.1
1
2 USE master;
3 GO
4
5 -- 1. Creación de logins para las cuentas de servicio de red (previamente creadas en Windows/Active Directory)
6 CREATE LOGIN [UNIVERSIDAD\svc_sqlengine_gt] FROM WINDOWS;
7 CREATE LOGIN [UNIVERSIDAD\svc_sqlagent_gt] FROM WINDOWS;
8 GO
9
10 -- 2. Asignación de permisos de nivel de servidor
11 ALTER SERVER ROLE sysadmin ADD MEMBER [UNIVERSIDAD\svc_sqlengine_gt];
12 ALTER SERVER ROLE sysadmin ADD MEMBER [UNIVERSIDAD\svc_sqlagent_gt];
13 GO
```

3. Justificación técnica de la solución Aunque la asignación real del servicio a la cuenta se realiza a nivel del sistema operativo mediante el *SQL Server Configuration Manager*, a nivel de base de datos es necesario aprovisionar los accesos. Se crean inicios de sesión basados en Windows para cuentas de servicio dedicadas (svc_sqlengine_gt y svc_sqlagent_gt) y se les asigna el rol sysadmin para que puedan ejecutar las tareas internas del motor y los trabajos programados.

4. Explicación de las buenas prácticas utilizadas

- **Aislamiento de Cuentas (Service Isolation):** Se evita el uso de cuentas genéricas como LocalSystem o NetworkService. Usar cuentas de dominio dedicadas limita el impacto si el servicio es vulnerado.
- **Trazabilidad:** Al tener cuentas separadas para el Motor (Engine) y el Agente (Agent), los registros de auditoría y los logs de errores de SQL Server mostrarán exactamente qué servicio ejecutó cada acción.

TEMA 2: Proyecto 2 - Monitoreo de servicios críticos del servidor

1. **Enunciado del ejercicio** Implemente consultas administrativas que permitan verificar el estado de las instancias SQL Server relacionadas con GradosTitulos.
2. Script de la solución en SQL

```
-- tema 2 .1
USE master;
GO

SELECT
    servicename,
    process_id,
    startup_type_desc,
    status_desc,
    last_startup_time,
    service_account
FROM sys.dm_server_services;
GO
```

Resultados		Mensajes				
	servicename	process_id	startup_type_desc	status_desc	last_startup_time	serv
1	SQL Server (MSSQLSERVER)	18588	Automatic	Running	2026-06-26 21:32:43.5792201 -05:00	NT
2	SQL Server Agent (MSSQLSERVER)	NULL	Manual	Stopped	NULL	NT

3. **Justificación técnica de la solución** Se consulta la Vista de Administración Dinámica (DMV) sys.dm_server_services. Esta vista devuelve información crítica del sistema operativo y de los nodos del clúster sobre los servicios de SQL Server y SQL Server Agent de la instancia actual, sin necesidad de salir del entorno T-SQL.
4. **Explicación de las buenas prácticas utilizadas**
 - **Monitoreo No Intrusivo:** El uso de DMVs es la forma más segura y rápida de obtener diagnósticos del servidor sin afectar el rendimiento ni bloquear recursos.
 - **Auditoría de Cuentas de Servicio:** Al incluir la columna service_account, se puede verificar rápidamente si los servicios se están ejecutando con las cuentas de red configuradas (como recomienda el Proyecto 1 del Tema 2).

TEMA 2: Proyecto 3 - Configuración de protocolos de red seguros

1. **Enunciado del ejercicio** Configure la instancia SQL Server para permitir únicamente conexiones seguras a la base de datos GradosTitulos.
2. Script de la solución en SQL

```

125 | --2.3
126 |
127 | USE GradosTitulos;
128 | GO
129 |
130 | -- 1. Revocar el acceso público a usuarios no autenticados
131 | REVOKE CONNECT FROM guest;
132 | GO
133 |
134 | -- 2. Consulta administrativa para auditar conexiones encriptadas en tiempo real
135 | SELECT
136 |     c.session_id,
137 |     c.net_transport,
138 |     c.auth_scheme,
139 |     c.client_net_address,
140 |     c.encrypt_option
141 | FROM sys.dm_exec_connections c
142 | JOIN sys.dm_exec_sessions s ON c.session_id = s.session_id
143 | WHERE s.database_id = DB_ID('GradosTitulos');
144 | GO

```

Resultados		Mensajes			
	session_id	net_transport	auth_scheme	client_net_address	encrypt_option
1	54	Shared memory	SQL	<local machine>	FALSE
2	93	Shared memory	SQL	<local machine>	FALSE

3. **Justificación técnica de la solución** Forzar la encriptación SSL/TLS es una tarea que se activa en el *SQL Server Configuration Manager* (Forzar cifrado = Sí). En este script complementario de T-SQL, se revoca explícitamente el permiso de conexión al usuario guest, bloqueando el acceso anónimo. Además, se proporciona una consulta cruzando sys.dm_exec_connections y sys.dm_exec_sessions para auditar que efectivamente la columna encrypt_option devuelva 'TRUE' para todos los que consultan GradosTitulos.
4. **Explicación de las buenas prácticas utilizadas**

- **Deshabilitación del usuario Guest:** Es una de las reglas de oro en *hardening* (endurecimiento) de bases de datos para evitar que usuarios sin mapeo explícito puedan acceder.
- **Auditoría Activa:** Utilizar Vistas de Administración Dinámica (DMVs) permite al DBA verificar que las configuraciones de red aplicadas en el servidor realmente están encriptando los paquetes de datos (protegiendo la información académica de ataques *Man-in-the-Middle*).

TEMA 3: Proyecto 1 - Roles para el proceso de grados y títulos

1. **Enunciado del ejercicio** Diseñe roles personalizados para gestionar el acceso a los esquemas: Catalogo, Academico, Tramite, Evaluacion, Documento.
2. Script de la solución en SQL

```

52 USE GradosTitulos;
53 GO
54
55
56 CREATE ROLE RolAdminCatalogos;
57 CREATE ROLE RolGestionAcademica;
58 CREATE ROLE RolCoordinadorTramites;
59 CREATE ROLE RolComiteEvaluacion;
60 CREATE ROLE RolEmisorDocumentos;
61 GO
62
63 GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Catalogo TO RolAdminCatalogos;
64 GRANT SELECT, INSERT, UPDATE ON SCHEMA::Academico TO RolGestionAcademica;
65 GRANT SELECT, INSERT, UPDATE ON SCHEMA::Tramite TO RolCoordinadorTramites;
66 GRANT SELECT, INSERT, UPDATE ON SCHEMA::Evaluacion TO RolComiteEvaluacion;
67 GRANT SELECT, INSERT ON SCHEMA::Documento TO RolEmisorDocumentos;
68 GO

```

3. **Justificación técnica de la solución** El script aborda la seguridad a nivel de esquema (Schema-level security). Se crea un rol específico por cada dominio de negocio de la base de datos. Al aplicar el comando GRANT directamente sobre el SCHEMA::[Nombre], cualquier tabla actual o futura creada dentro de ese esquema heredará automáticamente estas reglas de acceso.
4. Explicación de las buenas prácticas utilizadas
 - **Seguridad a Nivel de Esquema:** Reduce significativamente la cantidad de sentencias GRANT necesarias y minimiza la posibilidad de errores administrativos al agregar nuevas tablas en el futuro.
 - **Granularidad de Permisos:** Se restringe el comando DELETE a casi todos los roles. Por ejemplo, el emisor de documentos solo puede leer e insertar, asegurando que no se eliminen resoluciones o diplomas históricos.

TEMA 3: Proyecto 2 - Implementación de roles para evaluadores de tesis

1. **Enunciado del ejercicio** Los evaluadores deben consultar tesis y registrar observaciones sin modificar información académica.
2. Script de la solución en SQL

```

148 USE GradosTitulos;
149 GO
150
151 -- 1. Creación del rol
152 CREATE ROLE RolEvaluadorTesis;
153 GO
154
155 -- 2. Permisos de consulta sobre los trabajos de investigación
156 GRANT SELECT ON Tramite.TrabajoInvestigacion TO RolEvaluadorTesis;
157 GRANT SELECT ON Tramite.PlanTesis TO RolEvaluadorTesis;
158
159 -- 3. Permisos para registrar observaciones y evaluaciones
160 GRANT SELECT, INSERT, UPDATE ON Tramite.ObservacionTramite TO RolEvaluadorTesis;
161 GRANT SELECT, INSERT, UPDATE ON Evaluacion.EvaluacionTrabajo TO RolEvaluadorTesis;
162
163 -- 4. Bloqueo explícito de modificación en información académica base
164 DENY INSERT, UPDATE, DELETE ON SCHEMA::Academico TO RolEvaluadorTesis;
165 GO

```

3. **Justificación técnica de la solución** Se crea el rol RolEvaluadorTesis. Se le otorga permiso de lectura (SELECT) sobre las tablas donde residen las tesis, y permisos de escritura (INSERT, UPDATE) estrictamente sobre las tablas donde deben dejar sus revisiones. Finalmente, se utiliza el comando DENY a nivel del esquema Academico para garantizar que no puedan alterar datos de alumnos, facultades o matrículas.
4. **Explicación de las buenas prácticas utilizadas**
 - **Precedencia del comando DENY:** En SQL Server, un DENY siempre anula un GRANT. Si un evaluador pertenece accidentalmente a otro rol que sí permite editar el esquema académico, el DENY explícito bloqueará el acceso, protegiendo la integridad de la base de datos.
 - **Principio de Necesidad de Saber:** El evaluador recibe los permisos exactos para cumplir su flujo de trabajo (evaluar y observar) sin acceso a tablas administrativas o financieras (como `Tramite.Pago`), manteniendo la confidencialidad de la información del estudiante.

TEMA 3: Proyecto 3 - Administración de roles jerárquicos

1. **Enunciado del ejercicio** Configure roles que representen los niveles: Operador, Supervisor, Administrador para el sistema de grados y títulos.
2. Script de la solución en SQL

```
'1 USE GradosTitulos;
'2 GO
'3
'4
'5 CREATE ROLE NivelOperador;
'6 CREATE ROLE NivelSupervisor;
'7 CREATE ROLE NivelAdministrador;
'8 GO
'9
'10 GRANT SELECT, INSERT, UPDATE ON SCHEMA::Tramite TO NivelOperador;
'11
'12 ALTER ROLE NivelSupervisor ADD MEMBER NivelOperador;
'13 GRANT DELETE ON SCHEMA::Tramite TO NivelSupervisor;
'14 GRANT SELECT, UPDATE ON SCHEMA::Catalogo TO NivelSupervisor;
'15
'16 ALTER ROLE NivelAdministrador ADD MEMBER NivelSupervisor;
'17 GRANT CONTROL ON DATABASE::GradosTitulos TO NivelAdministrador;
'18 GO
```

3. **Justificación técnica de la solución** Se utiliza el anidamiento de roles (Role nesting) soportado en SQL Server. Se añaden roles dentro de otros roles usando ALTER ROLE ... ADD MEMBER El Supervisor hereda todos los permisos del Operador y añade la capacidad de eliminar registros y modificar catálogos. El Administrador hereda del Supervisor y recibe control total sobre la base de datos.
4. **Explicación de las buenas prácticas utilizadas**
 - **Herencia de Permisos:** Evita la duplicación de código y simplifica el mantenimiento. Si los permisos de lectura del operador cambian, el supervisor y el administrador heredan el cambio automáticamente.

- **Control Centralizado:** La asignación de GRANT CONTROL al rol administrador local previene la necesidad de otorgar el rol de servidor sysadmin a usuarios de la base de datos, manteniendo aislado el entorno de seguridad.

TEMA 4 — Control de Acceso mediante GRANT, DENY y REVOKE

Proyecto 1: Roles para el Proceso de Grados y Títulos

► 1. Enunciado

Los operadores del sistema deben poder consultar datos de estudiantes registrados en la base de datos GradosTítulos, pero no deben tener capacidad de modificarlos (INSERT, UPDATE, DELETE). Se requiere implementar este control de acceso usando roles de SQL Server con permisos granulares por esquema.

► 2. Script de Solución SQL

```

172 -- PASO 1: Crear el login de SQL Server (nivel servidor)
173 IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'LoginOperador')
174     CREATE LOGIN LoginOperador WITH PASSWORD = 'Oper@2024#Secure';
175 GO
176
177 -- PASO 2: Crear el usuario de base de datos
178 IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'UsuarioOperador')
179     CREATE USER UsuarioOperador FOR LOGIN LoginOperador;
180 GO
181
182 -- PASO 3: Crear rol específico de lectura operacional
183 IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_operador_lectura')
184     CREATE ROLE rol_operador_lectura;
185 GO
186
187 -- PASO 4: Asignar SELECT sobre esquemas relevantes
188 GRANT SELECT ON SCHEMA::Academico TO rol_operador_lectura;
189 GRANT SELECT ON SCHEMA::Tramite TO rol_operador_lectura;
190 GRANT SELECT ON SCHEMA::Catalogo TO rol_operador_lectura;
191 GO
192
193 -- PASO 5: DENY explícito de escritura (defensa en profundidad)
194 DENY INSERT ON SCHEMA::Academico TO rol_operador_lectura;
195 DENY UPDATE ON SCHEMA::Academico TO rol_operador_lectura;
196 DENY DELETE ON SCHEMA::Academico TO rol_operador_lectura;
197
198 DENY INSERT ON SCHEMA::Tramite TO rol_operador_lectura;
199 DENY UPDATE ON SCHEMA::Tramite TO rol_operador_lectura;
200 DENY DELETE ON SCHEMA::Tramite TO rol_operador_lectura;
201 GO
202
203 -- PASO 6: Agregar el usuario al rol
204 EXEC sp_addrolemember 'rol_operador_lectura', 'UsuarioOperador';
205 GO
206
207 -- VERIFICACIÓN: Comprobar permisos asignados
208 SELECT dp.name AS Principal, o.name AS Objeto,
209        p.permission_name, p.state_desc
210 FROM sys.database_permissions p
211 JOIN sys.database_principals dp ON dp.principal_id = p.grantee_principal_id
212 JOIN sys.schemas o ON o.schema_id = p.major_id
213 WHERE dp.name = 'rol_operador_lectura';

```

► 3. Justificación Técnica

Se implementa el principio de Menor Privilegio (Least Privilege): el operador recibe únicamente los permisos estrictamente necesarios para su función. El uso de DENY sobre INSERT/UPDATE/DELETE crea una segunda capa de seguridad (defensa en profundidad): incluso si el usuario fuera añadido accidentalmente a otro rol con mayor privilegio, el DENY explícito prevalecería sobre cualquier GRANT, impidiendo las operaciones de escritura.

Los permisos se asignan a nivel de SCHEMA en lugar de tabla individual, lo que garantiza que cualquier tabla nueva creada bajo dichos esquemas herede automáticamente la política de acceso.

✓ 4. Buenas Prácticas Utilizadas

- Aplicar el principio de Least Privilege: conceder solo lo mínimo necesario para cada rol.
- Usar DENY explícito como segunda capa de seguridad sobre los permisos de escritura.
- Gestionar permisos a nivel de SCHEMA para que apliquen automáticamente a tablas futuras.
- Usar roles en lugar de asignar permisos directamente al usuario, facilitando el mantenimiento.
- Contraseñas de login deben seguir política de complejidad (mayúsculas, números, caracteres especiales).
- Verificar los permisos asignados con `sys.database_permissions` tras cada cambio.

Proyecto 2: Restricción de Acceso a Resoluciones y Diplomas

► 1. Enunciado

Solo el personal de la Oficina de Grados y Títulos debe poder modificar los documentos oficiales almacenados en el esquema Documento (tablas Resolucion y Diploma). El resto de usuarios del sistema no debe tener acceso de escritura a este esquema bajo ninguna circunstancia.

► 2. Script de Solución SQL

```

-- Login y usuario para la Oficina de Grados y Títulos
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'LoginOficinaGT')
    CREATE LOGIN LoginOficinaGT WITH PASSWORD = 'OfiGT@2024#Doc';
GO

IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'UsuarioOficinaGT')
    CREATE USER UsuarioOficinaGT FOR LOGIN LoginOficinaGT;
GO

-- Rol exclusivo para Oficina de Grados y Títulos
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_oficina_gt')
    CREATE ROLE rol_oficina_gt;
GO

-- GRANT completo sobre esquema Documento
GRANT SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_oficina_gt;
-- También necesita leer Tramite para validaciones cruzadas
GRANT SELECT ON SCHEMA::Tramite TO rol_oficina_gt;
GRANT SELECT ON SCHEMA::Academico TO rol_oficina_gt;
GO

-- DENY de escritura para el resto de roles existentes
-- Evita que otros roles accedan al esquema Documento
DENY INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_operador_lectura;
GO

-- Crear rol generico de usuarios del sistema (sin acceso a Documento)
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_usuario_sistema')
    CREATE ROLE rol_usuario_sistema;
GO
GRANT SELECT ON SCHEMA::Academico TO rol_usuario_sistema;
GRANT SELECT ON SCHEMA::Tramite TO rol_usuario_sistema;
-- Sin ningún permiso sobre Documento
DENY SELECT, INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_usuario_sistema;
GO

-- Asignar usuario al rol exclusivo
EXEC sp_addrolemember 'rol_oficina_gt', 'UsuarioOficinaGT';
GO

-- VERIFICACIÓN
SELECT dp.name, p.permission_name, p.state_desc, o.name AS Esquema
FROM sys.database_permissions p
JOIN sys.database_principals dp ON dp.principal_id = p.grantee_principal_id
JOIN sys.schemas o ON o.schema_id = p.major_id
WHERE o.name = 'Documento';
GO

```

	name	permission_name	state_desc	Esquema
1	RolEmissorDocumentos	INSERT	GRANT	Documento
2	RolEmissorDocumentos	SELECT	GRANT	Documento
3	rol_operador_lectura	DELETE	DENY	Documento
4	rol_operador_lectura	INSERT	DENY	Documento
5	rol_operador_lectura	UPDATE	DENY	Documento
6	rol_oficina_gt	DELETE	GRANT	Documento
7	rol_oficina_gt	INSERT	GRANT	Documento
8	rol_oficina_gt	SELECT	GRANT	Documento
9	rol_oficina_gt	UPDATE	GRANT	Documento
10	rol_usuario_sistema	DELETE	DENY	Documento
11	rol_usuario_sistema	INSERT	DENY	Documento
12	rol_usuario_sistema	SELECT	DENY	Documento
13	rol_usuario_sistema	UPDATE	DENY	Documento

► 3. Justificación Técnica

El esquema Documento contiene datos críticos e irreversibles (resoluciones y diplomas oficiales). Aplicando el principio de Separación de Funciones, únicamente el rol rol_oficina_gt tiene permisos DML sobre este esquema. El uso de DENY sobre otros roles garantiza que incluso una mala configuración accidental no permita acceso a estos objetos. La lectura de los esquemas Tramite y Academico por parte de la Oficina GT es necesaria para las validaciones de integridad referencial en el proceso de emisión de resoluciones y diplomas.

✓ 4. Buenas Prácticas Utilizadas

- Aplicar Separación de Funciones (Segregation of Duties): cada rol accede solo a lo que le corresponde.
- Restringir el esquema Documento con DENY a todos los demás roles, no solo omitir el GRANT.
- Crear roles con nombres descriptivos que reflejen la función institucional del usuario.
- Conceder lectura de esquemas de apoyo (Tramite, Academico) solo cuando hay justificación operacional.
- Documentar en comentarios la razón de negocio de cada GRANT/DENY para facilitar auditorías futuras.

Proyecto 3: Revocación de Permisos Temporales para Auditor Externo

► 1. Enunciado

Un auditor externo requiere acceso de solo lectura a la base de datos GradosTitulos durante una semana. Al término del período el acceso debe ser revocado completamente. Se solicita el script de creación del acceso temporal y su posterior revocación, incluyendo el deshabilitar el login del servidor.

► 2. Script de Solución SQL

```

-- FASE 1: CREAMER ACCESO TEMPORAL (ejecutar al inicio)
-- Login de servidor con fecha de expiración (SQL Auth)
IF NOT EXISTS (SELECT 1 FROM sys.server_principals WHERE name = 'LoginAuditorExterno')
    CREATE LOGIN LoginAuditorExterno
        WITH PASSWORD = 'Audit@Ext2024#Tmp!',
            CHECK_EXPIRATION = ON,
            CHECK_POLICY = ON;

GO

-- Usuario de base de datos
CREATE USER UsuarioAuditorExterno FOR LOGIN LoginAuditorExterno;
GO

-- Rol de auditor: solo lectura total
IF NOT EXISTS (SELECT 1 FROM sys.database_principals WHERE name = 'rol_auditor_externo')
    CREATE ROLE rol_auditor_externo;
GO

-- Permisos de lectura en todos los esquemas (auditoria completa)
GRANT SELECT ON SCHEMA::Academico TO rol_auditor_externo;
GRANT SELECT ON SCHEMA::Tramite TO rol_auditor_externo;
GRANT SELECT ON SCHEMA::Evaluacion TO rol_auditor_externo;
GRANT SELECT ON SCHEMA::Documento TO rol_auditor_externo;
GRANT SELECT ON SCHEMA::Catalogo TO rol_auditor_externo;
GO

-- DENY de escritura (proteccion adicional)
DENY INSERT, UPDATE, DELETE ON SCHEMA::Academico TO rol_auditor_externo;
DENY INSERT, UPDATE, DELETE ON SCHEMA::Tramite TO rol_auditor_externo;
DENY INSERT, UPDATE, DELETE ON SCHEMA::Evaluacion TO rol_auditor_externo;
DENY INSERT, UPDATE, DELETE ON SCHEMA::Documento TO rol_auditor_externo;
GO

EXEC sp_addrolemember 'rol_auditor_externo', 'UsuarioAuditorExterno';
GO

-- Registrar fecha de creacion en tabla de control (recomendado)
-- INSERT INTO Administracion.AccesoTemporal VALUES
-- ('UsuarioAuditorExterno', SYSDATETIME(), DATEADD(DAY,7,SYSDATETIME()), 'Auditoria externa semana 26');
-- FASE 2: REVOCACIÓN COMPLETA (ejecutar al cabo de una semana)

-- Paso 1: Deshabilitar login inmediatamente (bloquea nuevas conexiones)
ALTER LOGIN LoginAuditorExterno DISABLE;
GO

-- Paso 2: Terminar sesiones activas del auditor
DECLARE @sql NVARCHAR(200);
DECLARE @spid INT;
DECLARE cur CURSOR FOR
    SELECT session_id FROM sys.dm_exec_sessions
    WHERE login_name = 'LoginAuditorExterno';
OPEN cur;
FETCH NEXT FROM cur INTO @spid;
WHILE @@FETCH_STATUS = 0
BEGIN
    SET @sql = N'KILL ' + CAST(@spid AS NVARCHAR);
    EXEC sp_executesql @sql;
    FETCH NEXT FROM cur INTO @spid;
END
CLOSE cur; DEALLOCATE cur;
GO

-- Paso 3: Revocar permisos del rol
REVOKE SELECT ON SCHEMA::Academico FROM rol_auditor_externo;
REVOKE SELECT ON SCHEMA::Tramite FROM rol_auditor_externo;
REVOKE SELECT ON SCHEMA::Evaluacion FROM rol_auditor_externo;

```

► 3. Justificación Técnica

El acceso temporal se modela en dos fases separadas y ejecutables de forma independiente: creación y revocación.

Se usa CHECK_EXPIRATION y CHECK_POLICY para forzar políticas de contraseña del servidor. La revocación en

cascada (KILL sesiones → REVOKE permisos → DROP usuario → DROP login) garantiza que no quede ningún vestigio

de acceso activo. El uso de REVOKE (en lugar de solo DENY) limpia completamente la entrada de permisos de

sys.database_permissions, dejando el catálogo de seguridad sin entradas residuales.

✓ 4. Buenas Prácticas Utilizadas

- Planificar la revocación desde el momento de creación del acceso temporal.
- Deshabilitar el LOGIN antes de hacer DROP, para cortar nuevas conexiones de forma inmediata.
- Terminar sesiones activas con KILL antes de eliminar el usuario para evitar errores de dependencia.
- Usar REVOKE en vez de solo DENY al terminar accesos temporales, para limpiar el catálogo de permisos.
- Registrar en una tabla de control los accesos temporales con fecha de inicio y expiración.
- Activar CHECK_POLICY y CHECK_EXPIRATION en logins temporales para reforzar la seguridad.

TEMA 5 — Cifrado y Protección de Datos (TDE y Always Encrypted)

Proyecto 1: Implementación de TDE para GradosTitulos

► 1. Enunciado

Se requiere proteger los archivos físicos (.mdf, .ldf, .bak) de la base de datos GradosTitulos mediante

Transparent Data Encryption (TDE). La encriptación debe ser completamente transparente para la aplicación

y los usuarios finales, sin requerir cambios en el código de acceso a datos.

► 2. Script de Solución SQL

```

-- PASO 1: Crear la clave maestra de la base de datos master
-- (solo si no existe)
IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name = '##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'MasterKey@GT2024#TDE!';
GO

-- PASO 2: Crear el certificado de servidor para TDE
IF NOT EXISTS (SELECT 1 FROM sys.certificates WHERE name = 'CertTDE_GradosTitulos')
    CREATE CERTIFICATE CertTDE_GradosTitulos
        WITH SUBJECT = 'Certificado TDE - Base GradosTitulos - UPLA',
        START_DATE = '20240101',
        EXPIRY_DATE = '20300101';
GO

-- PASO 3: Crear la clave de cifrado de base de datos (DEK)
USE GradosTitulos;
GO

CREATE DATABASE ENCRYPTION KEY
    WITH ALGORITHM = AES_256
    ENCRYPTION BY SERVER CERTIFICATE CertTDE_GradosTitulos;
GO

-- PASO 4: Activar TDE en la base de datos
ALTER DATABASE GradosTitulos
    SET ENCRYPTION ON;
GO

-- VERIFICACIÓN: Estado del cifrado
SELECT db.name,
       dek.encryption_state,
       CASE dek.encryption_state
           WHEN 0 THEN 'Sin cifrado'
           WHEN 1 THEN 'Sin cifrado (pendiente)'
           WHEN 2 THEN 'Cifrado en progreso'
           WHEN 3 THEN 'Cifrado ACTIVO'
           WHEN 4 THEN 'Cambio de clave'
           WHEN 5 THEN 'Descifrado en progreso'
       END AS EstadoCifrado,
       dek.percent_complete AS PorcentajeAvance,
       dek.key_algorithm,
       dek.key_length
FROM sys.dm_database_encryption_keys dek
JOIN sys.databases db ON db.database_id = dek.database_id
WHERE db.name = 'GradosTitulos';
GO

-- RESPALDO DEL CERTIFICADO (CRÍTICO - guardar en lugar seguro)
BACKUP CERTIFICATE CertTDE_GradosTitulos
    TO FILE = 'C:\Backups\CertTDE_GradosTitulos.cer'
    WITH PRIVATE KEY (
        FILE = 'C:\Backups\CertTDE_GradosTitulos.pvk',
        ENCRYPTION BY PASSWORD = 'Backup@Cert#2024!Secure'
    );
GO

```

► 3. Justificación Técnica

TDE cifra los datos a nivel de página de disco utilizando AES-256 (el estándar más robusto disponible en SQL Server).

La jerarquía de claves es: Service Master Key → Database Master Key → Certificado → DEK. Al activar TDE,

SQL Server cifra automáticamente todos los archivos físicos (MDF, LDF) y los backups generados a partir de

ese momento. La transparencia total significa que las aplicaciones, los stored procedures y las vistas no

requieren ninguna modificación. El respaldo del certificado es CRÍTICO: sin él es imposible restaurar la base

de datos en otro servidor, incluso con el backup .bak disponible.

✓ 4. Buenas Prácticas Utilizadas

- Usar AES_256 como algoritmo de cifrado (evitar AES_128 o 3DES por ser obsoletos).
- Respalidar inmediatamente el certificado TDE y su clave privada tras la creación.
- Guardar el backup del certificado en una ubicación física separada del servidor de BD.
- Monitorear sys.dm_database_encryption_keys para verificar que encryption_state = 3 (cifrado activo).
- Documentar la contraseña del respaldo del certificado en un gestor de contraseñas corporativo.
- Cifrar también la base de datos tempdb si la instancia contiene datos sensibles (SQL Server lo hace automáticamente cuando hay al menos una BD con TDE).

Proyecto 2: Protección de Datos Personales del Tesista con Always Encrypted

► 1. Enunciado

Se debe implementar Always Encrypted para proteger las columnas DNI, Correo y Telefono de la tabla

Academico.Persona. Los datos deben estar cifrados en tránsito y en reposo, de modo que ni los

administradores de base de datos puedan leer los valores en texto plano desde SQL Server Management Studio.

► 2. Script de Solución SQL

```

-- Verificar columnas actuales de Academico.Persona
USE GradosTitulos;
GO

-- Verificar columnas actuales de Academico.Persona
SELECT column_name, data_type, character_maximum_length
FROM INFORMATION_SCHEMA.COLUMNS
WHERE TABLE_SCHEMA = 'Academico' AND TABLE_NAME = 'Persona';
GO

-- Equivalente T-SQL (si CMK ya existe en almacén de claves):
CREATE COLUMN MASTER KEY CMK_GradosTitulos_Persona
WITH (
    KEY_STORE_PROVIDER_NAME = 'MSSQL_CERTIFICATE_STORE',
    KEY_PATH = 'CurrentUser/My/<thumbprint_del_certificado>'
);
GO

CREATE COLUMN ENCRYPTION KEY CEK_Persona_DatosPersonales
WITH VALUES
(
    COLUMN_MASTER_KEY = CMK_GradosTitulos_Persona,
    ALGORITHM = 'RSA_OAEP',
    ENCRYPTED_VALUE = 0x<valor_hex_generado_por_ssms>
);
GO

-- Tabla temporal con cifrado
CREATE TABLE Academico.PersonaEncriptada (
    IdPersona INT NOT NULL IDENTITY(1,1)
        CONSTRAINT PK_PersonaEnc PRIMARY KEY,

    -- DNI: Deterministic (permite búsquedas por igualdad)
    DNI CHAR(8) COLLATE Latin1_General_BIN2
        ENCRYPTED WITH (
            COLUMN_ENCRYPTION_KEY = CEK_Persona_DatosPersonales,
            ENCRYPTION_TYPE = DETERMINISTIC,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        ) NOT NULL,

    Nombres VARCHAR(80) NOT NULL,
    ApellidoPaterno VARCHAR(60) NOT NULL,
    ApellidoMaterno VARCHAR(60) NULL,

    -- Correo: Randomized (mayor seguridad, sin búsquedas directas)
    Correo VARCHAR(120) COLLATE Latin1_General_BIN2
        ENCRYPTED WITH (
            COLUMN_ENCRYPTION_KEY = CEK_Persona_DatosPersonales,
            ENCRYPTION_TYPE = RANDOMIZED,
            ALGORITHM = 'AEAD_AES_256_CBC_HMAC_SHA_256'
        ) NULL,

    -- Telefono: Randomized
    Telefono VARCHAR(15) COLLATE Latin1_General_BIN2

```

Resultados		Mensajes	
Columna	encryption_type_desc	encryption_algorithm_name	ClaveColumna

► 3. Justificación Técnica

Always Encrypted protege los datos sensibles de los tesisistas en dos capas: en reposo (archivos en disco) y en tránsito (entre servidor y cliente). A diferencia de TDE, el cifrado/descifrado ocurre en el driver del cliente (ADO.NET, JDBC), lo que significa que ni el DBA ni un atacante con acceso privilegiado al servidor pueden leer los valores. Se usa DETERMINISTIC para DNI (permite WHERE DNI = ?) y RANDOMIZED para Correo y Telefono.

Telefono (mayor entropía). El algoritmo AEAD_AES_256_CBC_HMAC_SHA_256 provee cifrado autenticado.

✓ 4. Buenas Prácticas Utilizadas

- Usar DETERMINISTIC solo en columnas donde se requieran búsquedas por igualdad (ej. DNI).
- Usar RANDOMIZED para columnas sin necesidad de filtros directos (mayor seguridad).
- La Column Master Key debe almacenarse en Azure Key Vault o en el almacén de certificados de Windows, nunca en la BD.
- Requiere COLLATE Latin1_General_BIN2 en columnas VARCHAR cifradas (obligatorio para Always Encrypted).
- Las aplicaciones cliente deben activar Column Encryption Setting=Enabled en el connection string.
- No se puede usar Always Encrypted con vistas indexadas, índices en columnas cifradas con RANDOMIZED, ni como FK.

Proyecto 3: Cifrado de Información Personal de Asesores y Jurados

► 1. Enunciado

Se requiere proteger la información personal de los docentes que actúan como asesores y jurados de tesis

(tabla Academico.Docente relacionada con Academico.Persona). La protección debe aplicarse mediante

cifrado simétrico a nivel de columna usando claves gestionadas dentro de SQL Server.

► 2. Script de Solución SQL

```

USE GradosTitulos;
GO

IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name = '##MS_DatabaseMasterKey##')
    CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'MasterKey@GT2024#Docentes!';
GO
IF NOT EXISTS (SELECT 1 FROM sys.certificates WHERE name = 'CertDocenteGT')
    CREATE CERTIFICATE CertDocenteGT
        WITH SUBJECT = 'Certificado cifrado datos Docentes UPLA';
GO

IF NOT EXISTS (SELECT 1 FROM sys.symmetric_keys WHERE name = 'SimKey_Docentes')
    CREATE SYMMETRIC KEY SimKey_Docentes
        WITH ALGORITHM = AES_256
        ENCRYPTION BY CERTIFICATE CertDocenteGT;
GO

ALTER TABLE Academico.Persona
    ADD CorreoCifrado VARBINARY(256) NULL,
        TelefonoCifrado VARBINARY(256) NULL;
GO

OPEN SYMMETRIC KEY SimKey_Docentes
    DECRYPTION BY CERTIFICATE CertDocenteGT;

UPDATE Academico.Persona
SET     CorreoCifrado = ENCRYPTBYKEY(KEY_GUID('SimKey_Docentes'), Correo),
        TelefonoCifrado = ENCRYPTBYKEY(KEY_GUID('SimKey_Docentes'), Telefono)
WHERE  TipoPersona = 'DOCENTE';

CLOSE SYMMETRIC KEY SimKey_Docentes;
GO

CREATE OR ALTER PROCEDURE Academico.SP_ObtenerDatosDocente
    @IdPersona INT
AS
BEGIN
    SET NOCOUNT ON;

    OPEN SYMMETRIC KEY SimKey_Docentes
        DECRYPTION BY CERTIFICATE CertDocenteGT;

    SELECT p.IdPersona,
           p.DNI,
           p.Nombres,
           p.ApellidoPaterno,
           CAST(DECRYPTBYKEY(p.CorrreoCifrado) AS VARCHAR(120)) AS Correo,
           CAST(DECRYPTBYKEY(p.TelefonoCifrado) AS VARCHAR(15)) AS Telefono
    FROM   Academico.Persona p
    WHERE  p.IdPersona = @IdPersona
           AND p.TipoPersona = 'DOCENTE';

    CLOSE SYMMETRIC KEY SimKey_Docentes;
END;
GO

```

► 3. Justificación Técnica

El cifrado simétrico a nivel de columna con AES-256 protege datos personales de docentes contra acceso no autorizado a nivel de SELECT directo sobre la tabla. La clave simétrica se almacena cifrada por el certificado, que a su vez está protegido por la Database Master Key. Solo los usuarios que tengan acceso al certificado podrán abrir la clave y descifrar los datos. Al encapsular la apertura/cierre de clave dentro de un stored procedure y restringir su ejecución con GRANT/DENY, se centraliza el control de acceso a datos sensibles.

✓ 4. Buenas Prácticas Utilizadas

- Encapsular la apertura y cierre de claves simétricas dentro de stored procedures para no exponer el proceso al cliente.
- Nunca dejar una symmetric key abierta fuera de la transacción que la necesita.
- Respalidar el certificado y la clave maestra igual que en TDE; su pérdida implica pérdida permanente de los datos.
- Almacenar las columnas cifradas como VARBINARY para evitar conversiones implícitas que puedan corromper el cifrado.
- Aplicar GRANT EXECUTE solo al rol autorizado y DENY explícito a los demás roles.
- Nunca usar ENCRYPTBYPASSPHRASE en producción: depende de contraseña hardcoded; preferir siempre claves gestionadas.

TEMA 6 — Auditoría y Monitoreo de Eventos con SQL Server Audit

Proyecto 1: Auditoría de Accesos a la Base de Datos

► 1. Enunciado

Se requiere registrar todos los accesos (conexiones exitosas y fallidas) realizados a la base de datos

GradosTitulos mediante SQL Server Audit. Los registros deben almacenarse en archivos de log seguros

y ser consultables para fines de cumplimiento normativo.

► 2. Script de Solución SQL

```

17 USE master;
18 GO
19
20 IF NOT EXISTS (SELECT 1 FROM sys.server_audits WHERE name = 'Audit_Accesos_GradosTitulos')
21 BEGIN
22     CREATE SERVER AUDIT Audit_Accesos_GradosTitulos
23     TO FILE (
24         FILEPATH          = 'C:\SQLAudit\GradosTitulos\',
25         MAXSIZE            = 100 MB,
26         MAX_ROLLOVER_FILES = 10,
27         RESERVE_DISK_SPACE = OFF
28     )
29     WITH (
30         QUEUE_DELAY      = 1000,    -- 1 segundo de latencia máxima
31         ON_FAILURE       = CONTINUE -- no detiene la BD si falla el log
32     );
33 END
34 GO
35
36 -- Activar la auditoría de servidor
37 ALTER SERVER AUDIT Audit_Accesos_GradosTitulos WITH (STATE = ON);
38 GO
39
40 USE GradosTitulos;
41 GO
42
43 IF NOT EXISTS (SELECT 1 FROM sys.database_audit_specifications WHERE name = 'AuditSpec_Accesos_GT')
44 BEGIN
45     CREATE DATABASE AUDIT SPECIFICATION AuditSpec_Accesos_GT
46     FOR SERVER AUDIT Audit_Accesos_GradosTitulos
47     ADD (DATABASE_OBJECT_ACCESS_GROUP), -- acceso a objetos de BD
48     ADD (DATABASE_LOGOUT_GROUP),        -- cierre de sesion en BD
49     ADD (USER_CHANGE_PASSWORD_GROUP)    -- cambio de contraseña
50     WITH (STATE = ON);
51 END
52 GO
53
54 USE master;
55 GO
56
57 IF NOT EXISTS (SELECT 1 FROM sys.server_audit_specifications WHERE name = 'AuditSpec_Login_GT')
58 BEGIN
59     CREATE SERVER AUDIT SPECIFICATION AuditSpec_Login_GT
60     FOR SERVER AUDIT Audit_Accesos_GradosTitulos
61     ADD (SUCCESSFUL_LOGIN_GROUP),        -- logins exitosos
62     ADD (FAILED_LOGIN_GROUP),           -- logins fallidos
63     ADD (LOGOUT_GROUP)                  -- logouts
64     WITH (STATE = ON);
65 END
66 GO
67
68 SELECT TOP 50
69     event_time,
70     action_id,
71     succeeded,
72     server_principal_name AS Usuario,
73     client_ip             AS IP,
74     database_name         AS BaseDatos,

```

► 3. Justificación Técnica

SQL Server Audit opera en dos niveles: SERVER AUDIT (define el destino y formato del log) y la especificación (define qué eventos se capturan). SUCCESSFUL_LOGIN_GROUP y FAILED_LOGIN_GROUP permiten detectar intentos de acceso no autorizados. El destino FILE es preferido sobre Application Log porque es más difícil de alterar y permite rotación automática de archivos. QUEUE_DELAY = 1000 balancea rendimiento con latencia de auditoría.

ON_FAILURE = CONTINUE garantiza que la BD no se detenga si el sistema de auditoría falla temporalmente.

✓ 4. Buenas Prácticas Utilizadas

- Almacenar los archivos de auditoría en un disco o partición diferente al de los datos de la BD.
- Establecer MAX_ROLLOVER_FILES para evitar que los logs consuman todo el espacio en disco.
- Capturar siempre FAILED_LOGIN_GROUP: los intentos fallidos son indicadores clave de ataques de fuerza bruta.
- Usar ON_FAILURE = CONTINUE en producción para no detener la BD; usar SHUTDOWN IMMEDIATELY solo en entornos de máxima seguridad.
- Consultar los logs regularmente con sys.fn_get_audit_file y generar reportes periódicos.
- Proteger la carpeta de auditoría con permisos NTFS restrictivos: solo la cuenta del servicio SQL Server debe escribir en ella.

Proyecto 2: Auditoría de Modificaciones en Trámites Académicos

► 1. Enunciado

Se requiere registrar todas las operaciones INSERT, UPDATE y DELETE realizadas sobre las tablas del esquema Tramite de la base de datos GradosTitulos. El objetivo es mantener un historial completo de cambios en los trámites académicos para garantizar la trazabilidad del proceso institucional.

► 2. Script de Solución SQL

```

USE master;
GO

IF NOT EXISTS (SELECT 1 FROM sys.server_audits WHERE name = 'Audit_DML_GradosTitulos')
BEGIN
    CREATE SERVER AUDIT Audit_DML_GradosTitulos
    TO FILE (
        FILEPATH          = 'C:\SQLAudit\GradosTitulos\DML\' ,
        MAXSIZE            = 200 MB,
        MAX_ROLLOVER_FILES = 20
    )
    WITH (
        QUEUE_DELAY = 1000,
        ON_FAILURE  = CONTINUE
    );
END
GO
ALTER SERVER AUDIT Audit_DML_GradosTitulos WITH (STATE = ON);
GO

USE GradosTitulos;
GO

IF NOT EXISTS (SELECT 1 FROM sys.database_audit_specifications WHERE name = 'AuditSpec_DML_Tramite')
BEGIN
    CREATE DATABASE AUDIT SPECIFICATION AuditSpec_DML_Tramite
    FOR SERVER AUDIT Audit_DML_GradosTitulos

    -- Tabla principal de trámites
    ADD (INSERT, UPDATE, DELETE ON Tramite.Tramite BY PUBLIC),

    -- Trabajo de investigación (tesis / suficiencia)
    ADD (INSERT, UPDATE, DELETE ON Tramite.TrabajoInvestigacion BY PUBLIC),

    -- Plan de tesis
    ADD (INSERT, UPDATE, DELETE ON Tramite.PlanTesis BY PUBLIC),

    -- Designaciones
    ADD (INSERT, UPDATE, DELETE ON Tramite.DesignacionAsesor BY PUBLIC),
    ADD (INSERT, UPDATE, DELETE ON Tramite.DesignacionJurado BY PUBLIC),

    -- Pagos
    ADD (INSERT, UPDATE, DELETE ON Tramite.Pago BY PUBLIC),

    -- Observaciones del trámite
    ADD (INSERT, UPDATE, DELETE ON Tramite.ObservacionTramite BY PUBLIC)

    WITH (STATE = ON);
END
GO

CREATE OR ALTER VIEW Tramite.V_AuditoriaDML
AS
SELECT
    af.event_time          AS FechaHora,
    af.action_id           AS TipoOperacion,
    af.server_principal_name AS UsuarioServidor,
    af.database_principal_name AS UsuarioRD

```

► 3. Justificación Técnica

La auditoría DML sobre el esquema Tramite garantiza trazabilidad completa de todas las modificaciones a los trámites académicos. Al especificar BY PUBLIC, cualquier usuario que realice una operación DML queda registrado, independientemente de su rol. La auditoría por tabla (no por esquema completo) permite afinar el granulado y evitar ruido de logs. La vista V_AuditoriaDML simplifica las consultas de investigación al filtrar por esquema y tipo de acción, proporcionando un interfaz limpio sobre los archivos de auditoría.

✓ 4. Buenas Prácticas Utilizadas

- Separar los archivos de auditoría DML de los de acceso (diferentes carpetas y SERVER AUDITs) para facilitar el análisis.
- Usar BY PUBLIC en lugar de especificar usuarios individuales para no perder eventos cuando se creen nuevos usuarios.
- Crear vistas de auditoría que filtren y presenten los datos de forma legible para los auditores.
- Establecer MAXSIZE generoso en tablas DML de alta actividad para evitar pérdida de eventos por log lleno.
- Incluir siempre el campo statement en las consultas de auditoría; contiene la sentencia SQL exacta ejecutada.
- Programar un job de SQL Server Agent que alerte cuando los archivos de auditoría superen el 80% del tamaño máximo.

Proyecto 3: Sistema Integral de Auditoría Institucional

► 1. Enunciado

Se requiere diseñar una solución de auditoría completa que registre: inicio de sesión, cambios de permisos, creación de usuarios, eliminación de objetos de base de datos y modificaciones en datos críticos.

La solución debe cubrir tanto el nivel de servidor como el nivel de base de datos.

► 2. Script de Solución SQL

```

USE master;
GO

-- =====
-- PASO 1: SERVER AUDIT centralizado para auditoría integral
-- =====
IF NOT EXISTS (SELECT 1 FROM sys.server_audits WHERE name = 'Audit_Institucional_GT')
BEGIN
    CREATE SERVER AUDIT Audit_Institucional_GT
    TO FILE (
        FILEPATH          = 'C:\SQLAudit\GradosTitulos\Institucional\',
        MAXSIZE            = 500 MB,
        MAX_ROLLOVER_FILES = 50,
        RESERVE_DISK_SPACE = OFF
    )
    WITH (
        QUEUE_DELAY = 1000,
        ON_FAILURE  = CONTINUE
    );
END
GO
ALTER SERVER AUDIT Audit_Institucional_GT WITH (STATE = ON);
GO

-- =====
-- PASO 2: Especificación de servidor (eventos de instancia)
-- =====
IF NOT EXISTS (SELECT 1 FROM sys.server_audit_specifications WHERE name = 'AuditSpec_Servidor_Integral')
BEGIN
    CREATE SERVER AUDIT SPECIFICATION AuditSpec_Servidor_Integral
    FOR SERVER AUDIT Audit_Institucional_GT

    -- 1. Inicio de sesión (exitosos y fallidos)
    ADD (SUCCESSFUL_LOGIN_GROUP),
    ADD (FAILED_LOGIN_GROUP),
    ADD (LOGOUT_GROUP),

    -- 2. Cambios de permisos a nivel servidor
    ADD (SERVER_PERMISSION_CHANGE_GROUP),

    -- 3. Creación / modificación / eliminación de logins
    ADD (LOGIN_CHANGE_PASSWORD_GROUP),
    ADD (SERVER_PRINCIPAL_CHANGE_GROUP),

    -- 4. Cambio de roles de servidor
    ADD (SERVER_ROLE_MEMBER_CHANGE_GROUP),

    -- 5. Operaciones DDL de servidor
    ADD (SERVER_OBJECT_CHANGE_GROUP)

    WITH (STATE = ON);
END
GO

-- =====
-- PASO 3: Especificación de base de datos (eventos de BD)
-- =====

```

```

USE GRADOSITILLOS;
GO

IF NOT EXISTS (SELECT 1 FROM sys.database_audit_specifications WHERE name = 'AuditSpec_BD_Integral')
BEGIN
    CREATE DATABASE AUDIT SPECIFICATION AuditSpec_BD_Integral
    FOR SERVER AUDIT Audit_Institucional_GT

    -- 1. Creación y modificación de usuarios de BD
    ADD (DATABASE_PRINCIPAL_CHANGE_GROUP),

    -- 2. Cambios de permisos dentro de la BD (GRANT, DENY, REVOKE)
    ADD (DATABASE_PERMISSION_CHANGE_GROUP),

    -- 3. Cambios en membresía de roles de BD
    ADD (DATABASE_ROLE_MEMBER_CHANGE_GROUP),

    -- 4. Eliminación de objetos (DROP TABLE, DROP VIEW, etc.)
    ADD (SCHEMA_OBJECT_CHANGE_GROUP),

    -- 5. Modificaciones de datos críticos: Diploma y Resolucion
    ADD (INSERT, UPDATE, DELETE ON Documento.Diploma BY PUBLIC),
    ADD (INSERT, UPDATE, DELETE ON Documento.Resolucion BY PUBLIC),

    -- 6. Cambios en datos de personas (incluye DBA directo)
    ADD (INSERT, UPDATE, DELETE ON Academico.Persona BY PUBLIC)

    WITH (STATE = ON);
END
GO

-- =====
-- PASO 4: Stored Procedure de consulta integral de auditoría
-- =====
CREATE OR ALTER PROCEDURE dbo.SP_ConsultarAuditoriaIntegral
    @FechaDesde DATETIME2 = NULL,
    @FechaHasta DATETIME2 = NULL,
    @Usuario VARCHAR(100) = NULL,
    @TipoEvento VARCHAR(20) = NULL
AS
BEGIN
    SET NOCOUNT ON;

    -- Valores por defecto: últimas 24 horas
    IF @FechaDesde IS NULL SET @FechaDesde = DATEADD(HOUR, -24, SYSDATETIME());
    IF @FechaHasta IS NULL SET @FechaHasta = SYSDATETIME();

    SELECT
        af.event_time          AS FechaHora,
        af.action_id           AS TipoEvento,
        af.class_type_desc     AS ClaseObjeto,
        af.server_principal_name AS LoginServidor,
        af.database_principal_name AS UsuarioBD,
        af.database_name       AS BaseDatos,
        af.schema_name         AS Esquema,
        af.object_name         AS Objeto,
        af.statement            AS SentenciaSQL,
        af.succeeded            AS Exitoso,
        af.client_ip            AS IPCliente,
        af.host_name            AS NombreHost

```

► 3. Justificación Técnica

La auditoría integral combina especificaciones de servidor y de base de datos bajo un único SERVER AUDIT, simplificando la gestión de archivos de log. Los grupos de auditoría de servidor (SERVER_PERMISSION_CHANGE_GROUP, SERVER_PRINCIPAL_CHANGE_GROUP) capturan cambios administrativos que no son visibles desde la BD. La

especificación de BD añade granularidad sobre objetos específicos (Diploma, Resolucion, Persona) que son los datos de mayor criticidad institucional. El stored procedure parametrizado SP_ConsultarAuditoriaIntegral proporciona una interfaz unificada para investigaciones de seguridad, sin exponer directamente sys.fn_get_audit_file.

✓ 4. Buenas Prácticas Utilizadas

- Centralizar múltiples especificaciones bajo un solo SERVER AUDIT reduce la fragmentación de archivos de log.
- Auditar SCHEMA_OBJECT_CHANGE_GROUP para detectar eliminaciones accidentales o maliciosas de tablas y vistas.
- DATABASE_ROLE_MEMBER_CHANGE_GROUP es crítico: alerta sobre escaladas de privilegios no autorizadas.
- Encapsular sys.fn_get_audit_file en un SP parametrizado para controlar el acceso y estandarizar las consultas.
- Agregar el campo host_name e IP en todas las consultas de auditoría para triaje forense.
- Revisar regularmente la especificación de auditoría: los grupos de eventos pueden cambiar entre versiones de SQL Server.
- Considerar exportar los logs de auditoría a un SIEM externo (Azure Sentinel, Splunk) para correlación de eventos.